



Microprocessor & Computer Architecture (μpCA)

UE24CS251B

Session - 6.1

Microprocessor & Computer Architecture (μpCA)



UE24CS251B

Multiplication Instruction

Syntax: $\text{MLA}\{\text{<cond>}\}\{\text{S}\} \text{ Rd, Rm, Rs, Rn}$
 $\text{MUL}\{\text{<cond>}\}\{\text{S}\} \text{ Rd, Rm, Rs}$

MLA	multiply and accumulate	$Rd = (Rm * Rs) + Rn$
MUL	multiply	$Rd = Rm * Rs$

Syntax: $\text{<instruction>\{\text{<cond>}\}\{\text{S}\} \text{ RdLo, RdHi, Rm, Rs}$

SMLAL	signed multiply accumulate long	$[RdHi, RdLo] = [RdHi, RdLo] + (Rm * Rs)$
SMULL	signed multiply long	$[RdHi, RdLo] = Rm * Rs$
UMLAL	unsigned multiply accumulate long	$[RdHi, RdLo] = [RdHi, RdLo] + (Rm * Rs)$
UMULL	unsigned multiply long	$[RdHi, RdLo] = Rm * Rs$

MULTIPLICATION – MUL - INSTRUCTION

- **MUL R0, R1, R2**
 - @ $R0 = (R1 \times R2)_{[31:0]}$
- Features:
 - Second operand can't be immediate
 - The result register must be different from the first operand
 - Cycles depends on core type
 - If S bit is set, C flag is meaningless

Multiplication Instruction

- There are 2 classes of multiply - producing 32-bit and 64-bit results
- 32-bit versions on an ARM/TDM will execute in 2 - 5 cycles
 - `MUL r0, r1, r2` ; `r0 = r1 * r2`
 - `MLA r0, r1, r2, r3` ; `r0 = (r1 * r2) + r3`
- 64-bit multiply instructions offer both signed and unsigned versions
 - For these instructions there are 2 destination registers
 - `[U|S]MULL r4, r5, r2, r3` ; `r5:r4 = r2 * r3`
 - `[U|S]MLAL r4, r5, r2, r3` ; `r5:r4 = (r2 * r3) + r5:r4`

Multiplication of Corresponding elements in an array

. DATA	Loop:
A: . word 1, 3, 5	LDR R1, [R5] , #4
B: . word 2, 4, 6	LDR R2, [R6] , #4
C: . word	MUL R0, R1, R2 ; multiply R1 And R2 And
. TEXT	Store Result In R0
LDR R5, =A	ADD R4, R4, R0
LDR R6, =B	STR R4, [R7] , #4
LDR R7, =C	SUB R3, R3, #1
MOV R3, #3 ; number of	TEQ R3, #0 ; Test For Equality
elements in array	BNE Loop
MOV R4, #0	SW 0x11 ; Interrupt For
	Termination Of Program

$$C = C + A[i] * B[i]$$

```

        .TEXT
00001000:E59F5030    LDR R5,=A
00001004:E59F6030    LDR R6,=B
00001008:E59F7030    LDR R7,=C
0000100C:E3A03003    Mov R3,#3        ;count Of Numbers
00001010:E3A04000    MOV R4,#0
00001014:           Loop:
00001014:E4951004           Ldr R1,[R5],#4
00001018:E4962004           Ldr R2,[R6],#4
0000101C:E0000291           Mul R0,R1,R2      ;multiple R1 And R2 And Store Result In R0
00001020:E0844000           Add R4,R4,R0
00001024:E4874004           STR R4,[R7],#4
00001028:E2433001           Sub R3,R3,#1
0000102C:E3330000           Teq R3,#0         ; Test For Equality
00001030:1AFFFFFF7           Bne Loop
00001034:EF000011           Swi 0x11          ; Interrupt For Termination Of Program

        .DATA
00001044:           A: .word 1,3,5
00001050:           B: .word 2,4,6
0000105C:           C: .word

```

MLA EXAMPLE – Matrix Representation in memory

0	00000000
1	00000004
2	
3	
4	
5	
6	
7	
8	00000020
9	

$$\begin{array}{c}
 \begin{array}{cccc}
 & 1 & 2 & 3 \\
 A = & 4 & 5 & 6 \\
 & 7 & 8 & 9
 \end{array}
 \Rightarrow
 \begin{array}{ccc}
 A(0,0) & A(0,1) & A(0,2) \\
 A(1,0) & A(1,1) & A(1,2) \\
 A(2,0) & A(2,1) & A(2,2)
 \end{array}
 \end{array}$$

Row Major Order for a 3 x 3 matrix :

$A[2, 2] = \text{Row Num} \times \text{No. of elements per row} + \text{Column Num}$

$$2 \times 3 + 2 =$$

8th location.

- Since each location is of 4 bytes, the address of the 3rd element in the 2nd row is
 $8 \times 4 = 32.$
- This is achieved in ARM processor, using MLA instruction as follows.

MLA R1, R2, R3,
R4

Memory Dst Row # # of elements per row
col #

- To get consecutive data elements in a @D Array.

Multiply-Accumulate $c = c + a[i] * b[i]$

<MLA> Rd, Rf, Rn, Rm @ $Rd = Rf \times Rn + Rm$

- **MLA** R4, R1, R2, R4 @ $R4 = R1 \times R2 + R4$

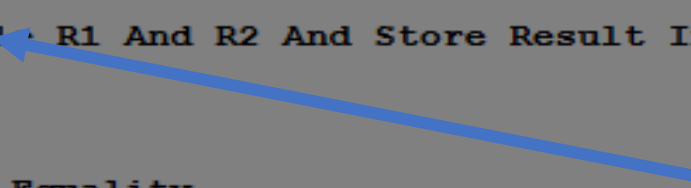
```

        .TEXT
00001000:E3A05D41    LDR R5,=A
00001004:E59F6028    LDR R6,=B
00001008:E59F7028    LDR R7,=C
0000100C:E3A03003    Mov R3,#3      ;count Of Numbers
00001010:E3A04000    MOV R4,#0
00001014:          Loop:
00001014:E4951004          Ldr R1,[R5],#4
00001018:E4962004          Ldr R2,[R6],#4
0000101C:E0244291    MLA R4,R1,R2,R4      ;multiply R1 And R2 And Store Result In R4

00001020:E4874004          STR R4,[R7],#4
00001024:E2433001          Sub R3,R3,#1
00001028:E3330000          Teq R3,#0          ; Test For Equality
0000102C:1AFFFFFF8      Bne Loop
00001030:EF000011          Swi 0x11          ; Interrupt For Termination Of Program

        .DATA
00001040:          A: .word 1,3,5
0000104C:          B: .word 2,4,6
00001058:          C: .word

```





Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 6.2

Conditional
Execution

Conditional Execution

- Almost all ARM instructions have a condition field which allows it to be executed conditionally.

MOVCS DB D1

Mnemonic	Condition	Mnemonic	Condition
CS	<i>Carry Set</i>	CC	<i>Carry Clear</i>
EQ	<i>Equal (Zero Set)</i>	NE	<i>Not Equal (Zero Clear)</i>
VS	<i>Overflow Set</i>	VC	<i>Overflow Clear</i>
GT	<i>Greater Than</i>	LT	<i>Less Than</i>
GE	<i>Greater Than or Equal</i>	LE	<i>Less Than or Equal</i>
PL	<i>Plus (Positive)</i>	MI	<i>Minus (Negative)</i>
HI	<i>Higher Than</i>	LO	<i>Lower Than (aka CC)</i>
HS	<i>Higher or Same (aka CS)</i>	LS	<i>Lower or Same</i>

Conditional Execution - Example 01

```
; Convert the C statement to ARM instructions.
if (R0!=5) {R1=R1+R0-R2}
```

```
CMP    R0, #5
BEQ    bypass
ADD    R1, R1, R0
SUB    R1, R1, R2
```

bypass: ...

```
CMP    R0, #5
ADDNE  R1, R1, R0
SUBNE  R1, R1, R2
```

; smaller and
faster

Rule of thumb: if the conditional sequence is three instructions or less, it is better to use conditional execution than a

Conditional Execution - Example 02

```
if ((R0==R1) && (R2==R3)) R4++
```

```
CMP    R0, R1
```

```
BNE    skip
```

```
CMP    R2, R3
```

```
BNE    skip
```

```
ADD    R4, R4, #1
```

```
skip: ...
```

```
CMP    R0, R1
```

```
CMPEQ  R2, R3
```

```
ADDEQ  R4, R4, #1
```

MCQs

1. How many destination registers are used by UMULL?

- A) 1
- B) 2
- C) 3
- D) 4

2. Which register pair stores the 64-bit result in UMULL R4, R5, R2, R3 ?

- A) R4 only
- B) R5 only
- C) R4:R5
- D) R5:R4

MCQs

1. How many destination registers are used by UMULL?

- A) 1
- B) 2**
- C) 3
- D) 4

2. Which register pair stores the 64-bit result in UMULL R4, R5, R2, R3 ?

- A) R4 only
- B) R5 only
- C) R4:R5
- D) R5:R4**

3. For a 3×3 matrix in row-major order, address of $A[2][2]$ is:

- A) 20 bytes
- B) 24 bytes
- C) 28 bytes
- D) 32 bytes

4. Why is conditional execution preferred for ≤ 3 instructions?

- A) Easier to write
- B) Reduces memory
- C) Smaller and faster
- D) Improves accuracy

3. For a 3×3 matrix in row-major order, address of $A[2][2]$ is:

- A) 20 bytes
- B) 24 bytes
- C) 28 bytes
- D) 32 bytes**

4. Why is conditional execution preferred for ≤ 3 instructions?

- A) Easier to write
- B) Reduces memory
- C) Smaller and faster**
- D) Improves accuracy

5. Conditional execution mainly reduces:

- A) Power consumption
- B) Clock cycles
- C) Branch penalty
- D) Register usage

MCQs

5. Conditional execution mainly reduces:

A) Power consumption

B) Clock cycles

C) Branch penalty

D) Register usage

THANK YOU



Team MPCA

Department of Computer Science
and Engineering